物理を組み込む機械学習

鉄鋼プロセスへの応用に向けて

村田 昇
早稲田大学 理工学術院

https://noboru-murata.github.io/talks/physics-informed-mi/2026-10-19/

- **フィジカルAI**
  現実世界で動く AI が目指すものと，物理の知識が担う役割
- **物理の組み込み方: 3 つのバイアス**
  3 つのバイアスと学習の枠組み，支配方程式，正則化の系譜
- **バイアスを実現する技術**
  3 つのバイアスの実現と，支える技術 (自動微分・数値積分・関数近似)
- **例題: 制約付き曲線の設計**
  道路の曲線を題材に，問題を少しずつ難しくして技術の必要性を確かめる
- **研究の流れとまとめ**
  技術の年表と研究の流れ，講演のまとめ

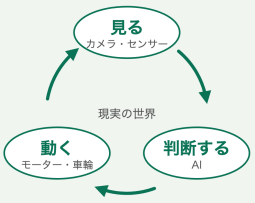
▸ **フィジカルAI**
物理の組み込み方: 3 つのバイアス
バイアスを実現する技術
例題: 制約付き曲線の設計
研究の流れとまとめ

フィジカルAIとは

定義

画面の中で文章や画像を扱うだけでなく，**現実の世界で，見て・判断して・動く** AI

- 家庭や工場で働くロボット，自動運転車，ドローン
- 2020 年ごろから学術的に提唱され (Miriyev & Kovač, 2020)，近年は産業界でも関心が高まっている



Miriyev, A., & Kovač, M. (2020). Skills for physical artificial intelligence. *Nature Machine Intelligence*, *2*(11), 658–660. ↗

何を実現したいのか

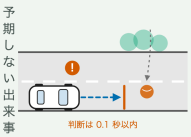
変化する環境で動く

変わっても通り道を決められる

人のそばで安全に働く

して人にぶつからずに動く

その場で素早く応じる



対応して止まる・よける

- 決まった場所で決まった作業を繰り返す従来のロボットから，**変化する現実の中で働ける** ロボットへ

実現に必要なもの

データと安全

動きのデータを集め，危ない動きをしないことを確かめる
(動作データ，安全の検証)

考える頭

映像と言葉から次の動きを決め，物の動きを予測する
(基盤モデル，世界モデル)[1]

仮想の練習場

コンピュータの中で何度も練習してから，実機に移す (シミュレーション，Sim-to-Real)

速く動く身体

力や触れた感じを捉え，その場で素早く制御する
(アクチュエータ，センサ，エッジ計算)

- 4 つの整理は学術・産業界の議論による．次の頁の 4 つの壁と対応[2]

Miriyev, A., & Kovač, M. (2020). Skills for physical artificial intelligence. *Nature Machine Intelligence*, *2*(11), 658–660. ↗
Thakur, A., Kaipa, K., et al. (2025). Physical Artificial Intelligence for Powering the Next Revolution in Robotics. *Journal of Computing and Information Science in Engineering*, *25*(12), 120809. ↗
1. Vision-Language-Action: カメラ画像と言葉の指示から，ロボットの動作の指令を直接出力する大規模モデル
2. Miriyev & Kovač (2020) の定義，Thakur et al. (2025)，および NVIDIA による産業界での整理 (学習・シミュレーション・実行の 3 つの計算機，NVIDIA Blog a) を参考にまとめた

立ちはだかる壁

データ: 集めるのが高い

- 実機の走行試験や測量にかかる時間と費用
- 転倒や衝突を伴う試行は機体を壊すので，危ない場面ほどデータが集まらない

安全: 法則を破る予測

- 物体が床をすり抜ける，曲がりきれない急カーブを指示する，といったありえない出力
- 珍しい状況ほどデータが少なく，予測が外れやすい場面

ずれ: 仮想と現実の違い

- シミュレーションで覚えた歩き方や持ち方が，実機では通用しない (転倒・落下)
- 路面の摩擦，タイヤの特性，機体の質量などの小さな違いの積み重ね

速さ: 一瞬で判断する

- 制御の周期は数〜数十 ms．その間に予測と判断
- 機体に積める計算機は小さく，電力も限定．大きなモデルはそのままでは載らない

Physics-Informed Machine Learning

物理を組み込む機械学習 (PIML)

物理法則をあらかじめ「知識」として学習に組み込み，データと法則の両方から学ぶ (Karniadakis et al., 2021)

"Physics-Informed" は「物理(法則)を組み込んだ」という意味

壁	PIML による答え	例題
データ: 高くつく	**少ないデータでも学べる**	問題3
安全: 法則を破る予測	**ありえない動きを出しにくい**	問題4・7
ずれ: 仮想と現実	**未知の状況でも，もっともらしく予測する**	問題3
速さ: 一瞬の判断	**学習済みの模型で，速く予測する**	問題5

Karniadakis, G. E., Kevrekidis, I. G., et al. (2021). Physics-informed machine learning. *Nature Reviews Physics*, *3*(6), 422–440. ↗

鉄鋼プロセスでも同じ壁

壁	鉄鋼プロセスでは (例)	例題
データ: 高くつく	材料の内部の温度や組織は直接測れず，測れるのは表面の数点	問題3
安全: 法則を破る予測	熱や質量の収支を破る予測は，操業の判断に使えない	問題4・7
ずれ: 仮想と現実	熱伝達係数や変形抵抗などの係数が，設備や操業の条件で変わる	問題3
速さ: 一瞬の判断	圧延などの設定計算や制御は，短い周期で答えを出す必要がある	問題5

- 伝熱や変形の **方程式は分かっている** が，係数や境界の条件の一部が **分からない**
  → PIML が働きやすい状況

9 / 83

フィジカルAI
▸ **物理の組み込み方: 3 つのバイアス**
バイアスを実現する技術
例題: 制約付き曲線の設計
研究の流れとまとめ

10 / 83

物理の組み込み方は 3 つ

観測バイアス

データ に組み込む
- 何を見せるか
- 物理を反映した観測・シミュレーション・データ拡張
- 満たすのはデータのある範囲だけ

学習バイアス

学習の目標 に組み込む
- 何を良しとするか
- 方程式のずれを罰則に
- 罰則の分だけ満たす \\ (ソフト制約)

帰納バイアス

学習器の形 に組み込む
- 何を表せるか
- 作れるのは物理を満たす関数だけ
- 常に満たす \\ (ハード制約)

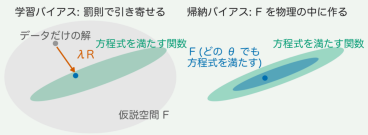
- **バイアス** は「偏見」ではなく，効率よく学ぶために与える **好ましい偏り** (先入観・事前の仮定)．解の候補を絞り込む制約として働く (Karniadakis et al., 2021)

Karniadakis, G. E., Kevrekidis, I. G., et al. (2021). Physics-informed machine learning. *Nature Reviews Physics*, *3*(6), 422–440. ↗

11 / 83

学習の枠組みと 3 つのバイアス

$$\underbrace{\min_{f \in \mathcal{F}}}_{\text{帰納バイアス}\atop\text{仮説空間}} \underbrace{\frac{1}{n}\sum_{k=1}^{n} \ell\big(f(x_k), y_k\big)}_{\text{観測バイアス}\atop\text{データ}} + \lambda \underbrace{\mathcal{R}[f]}_{\text{学習バイアス}\atop\text{正則化}}$$



学習の枠組みは **そのまま**．物理を組み込む場所が 3 つあるだけ
→ PIML は古典的な学習の **素直な拡張**

12 / 83

支配方程式と境界条件

- 物理法則の多くは **微分方程式** で書ける
- 解 u が満たす式を $\mathcal{N}[u;\lambda]=0$，境界・初期での条件を $\mathcal{B}[u]=g$ と書く

例	支配方程式 $\mathcal{N}[u;\lambda]=0$	境界・初期条件 $\mathcal{B}[u]=g$	物理パラメータ λ
熱方程式	$\partial_t u - \alpha\,\partial_x^2 u = 0$	$u(x,0)=u_0(x)$ $u(0,t)=u(1,t)=0$	熱拡散率 α
ばね・質量	$m\ddot{u}+c\dot{u}+ku=F(t)$	$u(0)=u_0,\ \dot{u}(0)=v_0$	m, c, k
曲線 (例題)	$x'-\cos\varphi=0,\ y'-\sin\varphi=0$ $\varphi'-\kappa=0$	$(x,y,\varphi)(0)=(0,0,0)$ 終点の条件	曲率 $\kappa(s)$

- 解 u は時刻や位置 x の関数．$\mathcal{N}$ は導関数を含む (自動微分で計算する)
- 境界条件・初期条件がなければ解は一つに決まらない

13 / 83

正則化の系譜

罰則 (Tikhonov) → 学習バイアス

$$\min_f \|Af-y\|^2+\lambda\|Lf\|^2$$

- 正則化項を損失に足す．満たし方は λ しだい (ソフト制約)
- $L=\mathcal{N}$ とすると PINN の物理残差

集合の制限 (Ivanov) → 帰納バイアス

$$\min_f \|Af-y\|^2 \quad \text{s.t. } \|Lf\|\le\tau$$

- 解の候補を集合に限る．$L=\mathcal{N}$，$\tau=0$ で「方程式をちょうど満たす関数の集合」(ハード制約)
- λ と τ はラグランジュ乗数で対応 (Tikhonov & Arsenin, 1974/1977)

- 正則化項の移り変わり: **パラメータ** の大きさ $\|\theta\|^2$ → **関数の微分** $\int f''(x)^2dx$ (Moody, 1991; Wahba, 1990) → **方程式** $\|\mathcal{N}[u_\theta]\|^2$ (PINN)

関数に条件を課す正則化は古くからの考え．新しいのは，学習器の微分や積分を自由に計算できるようになったこと

14 / 83

フィジカルAI
物理の組み込み方: 3 つのバイアス
▸ **バイアスを実現する技術**
例題: 制約付き曲線の設計
研究の流れとまとめ

15 / 83

観測バイアスの実現: データに組み込む

物理を反映したデータ
- 実験・観測のデータ
- シミュレーションで作ったデータ (代わりの模型を学ぶ)
- 対称性によるデータ拡張: 回転・平行移動・反転しても法則は変わらない

例題では
- 測量点 (x_k, y_k) がデータ項 (問題3)
- 曲線を左右に反転した曲線も，やはり正しい曲線 (曲率の符号を反転)

- 学習の仕組みには手を加えないので，いちばん手軽
- ただし法則が守られるのは **データのある範囲だけ**．外挿では物理法則が崩れる (問題3)

16 / 83

学習バイアスの実現: 方程式のずれを罰則に

物理残差を正則化項に

$$\mathcal{R}[u_\theta] = \int_\Omega |\mathcal{N}[u_\theta](x)|^2 dx \approx \frac{1}{n_r}\sum_{i=1}^{n_r} |\mathcal{N}[u_\theta](x_i)|^2$$

- $\mathcal{N}$ の中の導関数 → **自動微分**
- 積分 → **数値積分** (点 x_i での和)
- θ についての勾配が計算できる

例題では (曲線の方程式)

学習器が解をそのまま表す場合:

$f_\theta : t \mapsto (x, y, \varphi),\ u_\theta = f_\theta.$

$$r = (x' - L\cos\varphi,\ y' - L\sin\varphi,\ \varphi' - L\kappa)$$

$$\mathcal{R} = \frac{1}{n_r}\sum_i |r(t_i)|^2$$

- 方程式は罰則の分だけ満たされる
- 満たし方は重みと学習の進み方次第

- 学習器はそのまま，損失に **方程式のずれ** を足すだけ: PINN (物理を組み込んだ NN) (Raissi et al., 2019)

Raissi, M., Perdikaris, P., & Karniadakis, G. E. (2019). Physics-informed neural networks: A deep learning framework for solving forward and inverse problems involving nonlinear partial differential equations. *Journal of Computational Physics, 378*, 686–707. ↗

自動微分: 学習した関数の導関数

- 支配方程式は導関数を含む → 残差の評価に $u'_\theta,\ u''_\theta$ が要る
- ニューラルネットワーク (NN) は単純な関数の合成 → 連鎖律で，入力 x についての導関数が **厳密に・機械的に** 求まる
- NN のパラメータ θ (重みとバイアス) の勾配と同じ仕組みを入れ子で使う
- **積分** を含むとき (例: 曲率 → 方位 → 位置) は，関数値の有限和 (数値積分) で計算．和も微分できるので学習が可能

```python
# u'' = f の残差
from torch.autograd import grad
x.requires_grad_(True)
u = net(x)
g = dict(create_graph=True,
         grad_outputs=torch.ones_like(
du  = grad(u,  x, **g)[0]
d2u = grad(du, x, **g)[0]
loss = ((d2u - f(x))**2).mean()
loss.backward()  # θ の勾配

# 積分: 累積和も微分可能
phi = torch.cumsum(kappa, 0) * ds
```

PyTorch の例．net を入力 x について 2 回微分して残差 $u'' - f$ を作り (create_graph=True で微分の計算もグラフに残す．grad_outputs で各点の導関数をまとめて求める)，損失を θ について逆伝播する．最後の行: 積分を累積和で書けば，これも微分できる

NN を入力について微分できる
→ 方程式のずれを損失にできる

損失の組み立て: 点ごとのずれの和

$$\mathcal{L}_r = \int_\Omega |\mathcal{N}[u_\theta](x)|^2 dx \approx \frac{1}{n_r}\sum_{i=1}^{n_r} |\mathcal{N}[u_\theta](x_i) - 0|^2 \quad \text{方程式}$$

$$\mathcal{L}_b = \int_{\partial\Omega} |\mathcal{B}[u_\theta](x) - g(x)|^2 dx \approx \frac{1}{n_b}\sum_{j=1}^{n_b} |\mathcal{B}[u_\theta](x_j) - g_j|^2 \quad \text{境界}$$

$$\mathcal{L}_d = \mathbb{E}\,|u_\theta(x) - y|^2 \approx \frac{1}{n_d}\sum_{k=1}^{n_d} |u_\theta(x_k) - y_k|^2 \quad \text{データ}$$

- 方程式の損失も「点の上の誤差の平均」．x_i: コロケーション点 (残差を評価する点)
- 方程式・境界・観測の 3 項を同じ学習の枠組みで (確率的勾配法)

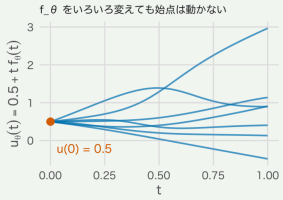
コロケーション点は「残差 0」を観測した仮想のデータ
→ 方程式・境界・観測を同じ枠組みで

帰納バイアスの実現 1: 出力の形で満たす

固定の写像を学習器の後ろに置く

$$u_\theta = \Phi(f_\theta)$$

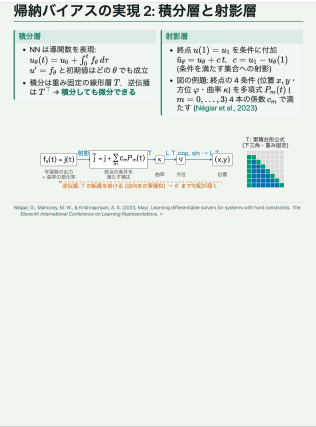
- f_θ: 学習器，Φ: 固定の写像
- 試行解 (trial solution) $u_\theta = g + \phi\, f_\theta$ (Lagaris et al., 1998)
  - g: 境界条件を満たす，ϕ: 境界で 0
  - どの θ でも境界条件が成立



- 例: 始点なら $g(0) = u_0,\ \phi = t$ (右図は $g = u_0$ の定数)．両端なら $\phi = t(1-t)$．複雑な領域では ϕ に距離関数 (Sukumar & Srivastava, 2022)

Lagaris, I. E., Likas, A., & Fotiadis, D. I. (1998). Artificial neural networks for solving ordinary and partial differential equations. *IEEE Transactions on Neural Networks, 9*(5), 987–1000. ↗
Sukumar, N., & Srivastava, A. (2022). Exact imposition of boundary conditions with distance functions in physics-informed deep neural networks. *Computer Methods in Applied Mechanics and Engineering, 389*, 114333. ↗

帰納バイアスの実現 2: 積分層と射影層



帰納バイアスの実現 3: ハミルトン力学

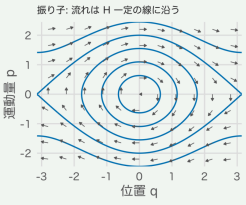
HNN: エネルギーを学ぶ

NN がエネルギー $H_\theta(q,p) = f_\theta(q,p)$ を表し，運動を

$$\dot{q} = \frac{\partial H_\theta}{\partial p}, \qquad \dot{p} = -\frac{\partial H_\theta}{\partial q}$$

で作って時間積分 (Greydanus et al., 2019)

- 流れは H_θ の等高線に沿う
  → **エネルギーが保存**
- 学習: 観測した速度 $(\dot{q}, \dot{p})$ とのずれを最小化



Greydanus, S., Dzamba, M., & Yosinski, J. (2019). Hamiltonian Neural Networks. *Advances in Neural Information Processing Systems, 32*. ↗

関数の学習と作用素の学習

関数を学ぶ (PINN)

$$x \longmapsto u_\theta(x)$$

- 条件 a (係数・境界条件・目標) を **1 つに固定**
- 学ぶのは 1 つの解 u
- 条件が変われば学び直し
- 例題の問題1〜4 (1 件 9〜70 秒)

作用素を学ぶ (NO / PINO)

$$a \longmapsto u_\theta = \mathcal{G}_\theta[a]$$

- 条件 a (数値や **関数**) も学習器の入力にとる: $\mathcal{G}_\theta[a](x) = f_\theta(a, x)$
- 学ぶのは「条件 → 解」の **対応**
- 学習は重いが，1 回の順伝播で新しい a に対する解を計算
- 例題の問題5 (終点の条件 → 曲率，0.7 ms)，問題6 (折れ線 → 滑らかな曲線)

- 入力も出力も **関数** になりうる (関数から関数への写像)．損失の形は同じで，違いは条件 a について平均をとるかどうか

関数近似: 関数と作用素の万能近似

関数を近似する

$$u(x) \approx \sum_{k=1}^{n} c_k\, \sigma(w_k \cdot x + b_k)$$

- 1 隠れ層でも，連続関数を任意の精度で近似可能 (Cybenko, 1989; Hornik et al., 1989)
- 解 $u(x)$ を NN で表す根拠 (Physics-Informed NN; PINN)

作用素を近似する

$$\mathcal{G}[a](x) \approx \sum_{k=1}^{p} \underbrace{b_k(a)}_{\text{係数}}\ \underbrace{t_k(x)}_{\text{場所の関数}}$$

- 係数 = 入力関数との積分 (重み付き平均) (Chen & Chen, 1995)
- a を表す点の数によらない (CNN・Transformer と同様)
- 条件 → 解の写像を NN で表す根拠 (Neural Operator; NO)

どちらも **表せること** の保証．必要なユニット数や，学習でその近似に到達できるかは別の問題

学習器 f_θ は何を表すか

- 求めたいのは u_θ，学習するのは f_θ
- 学習器 f_θ が系の **どの関数** を表し，求める u_θ を **どう作るか** が方法の違い

方法	学習器 f_θ が表すもの	求めたい関数 u_θ の作り方	例題
PINN (学習バイアス)	解そのもの $x \mapsto u$	$u_\theta = f_\theta$．方程式は罰則で	—
試行解	境界からのずれ	$u_\theta = g + \phi f_\theta$．境界条件を形で	問題1〜3
積分層＋射影層	導関数 u' (高階なら最高階)	f_θ を射影して積分 → u	問題4・7b
条件付き PINN	条件ごとの解: $(x, a) \mapsto u$	$u_\theta = f_\theta(\cdot, a)$．例題は積分層と組み合わせて u' を学ぶ	問題5
FNO (作用素)	関数 → 関数: $a \mapsto u$	1 回の順伝播で解．例題は u' を出して積分 (射影は推論時だけ)	問題6
HNN	エネルギー $H(q,p)$	勾配 $(\partial_p H, -\partial_q H)$ を時間積分 → 運動 $(q,p)(t)$	—

- 帰納バイアスは $u_\theta = \Phi(f_\theta)$ の Φ (学習しない固定の写像) の選び方
- 学習バイアスは損失の選び方

25 / 83

PINN と PINO の定式化

- 解を NN で表す: **PINN** は $u_\theta(x)$，**PINO** (物理を組み込んだ作用素学習) は条件 a も入力にとり $u_\theta(x) = \mathcal{G}_\theta[a](x)$

PINN と PINO の損失 (統一の形)

$$\mathcal{L}(\theta, \lambda) = \mathbb{E}_a \Big[\underbrace{w_r \frac{1}{n_r} \sum_{i=1}^{n_r} \left| \mathcal{N}[u_\theta; \lambda, a](x_i) \right|^2}_{\text{方程式のずれ } \mathcal{L}_r} + \underbrace{w_b \frac{1}{n_b} \sum_{j=1}^{n_b} \left| \mathcal{B}[u_\theta](x_j) - g_j(a) \right|^2}_{\text{境界・初期条件のずれ } \mathcal{L}_b} + \underbrace{w_d \frac{1}{n_d} \sum_{k=1}^{n_d} \left| u_\theta(x_k) - y_k \right|^2}_{\text{観測とのずれ } \mathcal{L}_d} \Big]$$

- PINN は条件 a を 1 つに固定した場合 ($\mathbb{E}_a$ がない)
- PINO は条件の分布で平均し，族全体を一度に学ぶ

26 / 83

何を求めるか: 順問題・設計問題・逆問題

- 損失の形は同じまま，**何が分かっていて何を求めるか** で 3 種類の問題

問題	分かっているもの	求めるもの (PINN)	損失に入れる項	PINO での条件 a
順問題	方程式・λ・境界条件	解 u	方程式＋境界	λ・境界条件・入力関数
設計問題	方程式の形・目標の条件	解 u と λ (設計変数)	方程式＋目標	目標の条件
逆問題 (同定)	方程式の形・観測データ	解 u と未知の λ	方程式＋観測	観測データ

- PINO: 表の「分かっているもの」を条件 a として入力にとり，$a \mapsto (u, \lambda)$ をまとめて学ぶ

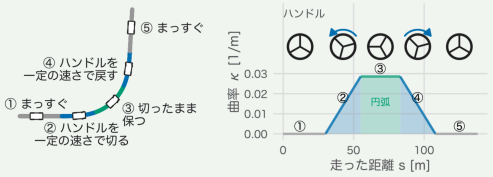
> 未知の λ は θ と **一緒に** 勾配法で更新するだけ
> → 順問題・設計・同定が同じ損失 $\mathcal{L}(\theta, \lambda)$ で解ける

27 / 83

フィジカルAI
物理の組み込み方: 3 つのバイアス
バイアスを実現する技術
▸ **例題: 制約付き曲線の設計**
研究の流れとまとめ

28 / 83

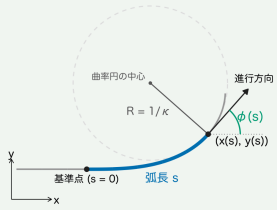
運転の基本操作



- カーブは **切る → 保つ → 戻す** の組合せで曲がる
- 曲がり具合 (曲率) はハンドルの切れ角にほぼ比例

29 / 83

記号の定義



記号	意味
s	弧長: 基準点からの道のり [m]
$(x(s), y(s))$	弧長 s の点の位置 [m]
$\varphi(s)$	方位角: 進行方向の角 [rad]
$\kappa(s)$	曲率: 曲がり具合 $= 1/R$ [1/m]

- 左曲がりで $\kappa > 0$，直線で $\kappa = 0$

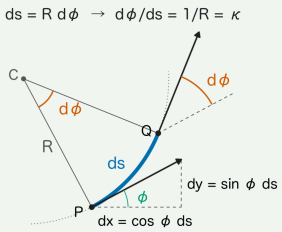
30 / 83

曲線の微分方程式

曲線の方程式

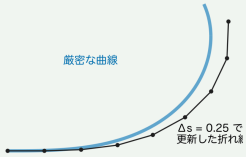
$$\frac{dx}{ds} = \cos\varphi(s)$$
$$\frac{dy}{ds} = \sin\varphi(s)$$
$$\frac{d\varphi}{ds} = \kappa(s)$$

- 曲率 (ハンドル操作) を与えると，位置と向き (軌跡) が決まる



31 / 83

積分して経路を作る



- 例: $\kappa(s) = s$ の曲線 (左図)
- 始点から刻み Δs ごとに更新

$$x_{i+1} = x_i + \cos\varphi_i \, \Delta s$$
$$y_{i+1} = y_i + \sin\varphi_i \, \Delta s$$
$$\varphi_{i+1} = \varphi_i + \kappa_i \, \Delta s$$

- $\Delta s \to 0$ の極限が積分
- 曲率と始点から経路を求める: **順問題**

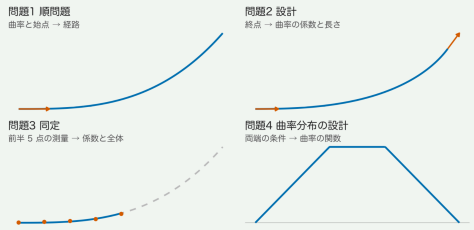
32 / 83

曲率による分類

区間	ハンドル	曲率 $\kappa(s)$	方位角 $\varphi(s)$
直線	中立のまま固定	0	一定
クロソイド	一定の速さで切っていく	s/A^2	$s^2/(2A^2)$
円弧	同じ角度を保つ	$1/R$	s/R

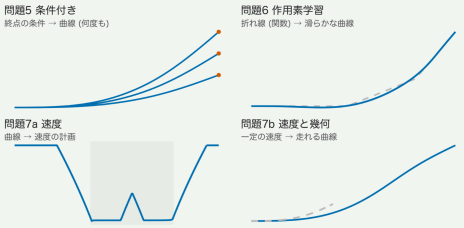
- **クロソイド**: 曲率が弧長に比例する曲線 (運転の基本操作の②④)
- 半径 R の円弧につなぐ長さ L のクロソイドは $A^2 = RL$
- $x(s) = \int_0^s \cos\left(u^2/2A^2\right) du$ は初等関数で書けない → **数値積分が要る**

8 つの問題: 少しずつ難しく (1)



曲線 (問題1) → 数個の係数 (問題2・3) → 曲率の関数そのもの (問題4)

8 つの問題: 少しずつ難しく (2)



何度も解く (問題5・6) から，走れるかどうかを含めた設計 (問題7a・7b) へ

問題ごとの道具

問題	与えるもの → 求めるもの	前の方法で困ること	新しい道具	講演
1	曲率と始点 → 経路	—	PINN の基本形	付録
2	終点 → 曲率の係数と長さ	未知の係数がある	係数も NN と同時に学習	付録
3	前半 5 点の測量 → 係数と全体	観測が一部の区間にしかない	データの項と，その重み	**本編**
4	両端の条件 → 曲率の関数	未知が関数．罰則では端と方程式がずれる	積分層と射影 (構造で満たす)	**本編**
5	終点の条件 → 曲線 (何度も)	1 件ごとに数十秒かかる	条件付き PINN	**本編**
6	折れ線 (関数) → 滑らかな曲線	入力が関数で，点数もまちまち	作用素学習 (FNO)	**本編**
7a	曲線 → 速度	曲線だけでは横加速度などが上限を超える	凸最適化 (NN は使わない)	付録
7b	一定の速度 → 走れる曲線	7a では減速が要る	動特性の罰則	付録

本編は 問題3 (データ)・問題4 (安全)・問題5・6 (速さ)．なお問題1 の答えは「方程式も入力も分かっている順問題なら数値積分で十分」

例題の損失関数

例題の損失の項

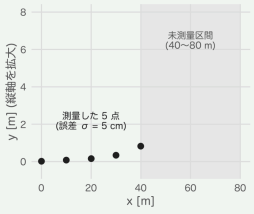
残差	$r(t) = \left(x' - L\cos\varphi,\ y' - L\sin\varphi,\ \varphi' - L\kappa\right)$	
方程式	$\mathcal{L}_r = \dfrac{1}{n_r}\sum_{i=1}^{n_r} \lvert r(t_i)\rvert^2$	
境界	$\mathcal{L}_b = \lvert (x,y,\varphi)(1) - (x_1,y_1,\varphi_1)\rvert^2$	終点 (始点は構造で満たす)
データ	$\mathcal{L}_d = \dfrac{1}{n_d}\sum_{k=1}^{n_d} \lvert (x,y)(t_k) - (x_k,y_k)\rvert^2$	
滑らかさ	$\mathcal{L}_s = \dfrac{1}{n_r}\sum_{i=1}^{n_r} \lvert\kappa''(t_i)\rvert \approx \int_0^1 \lvert\kappa''\rvert\, dt$	L1 (絶対値の和．区分線形を好む)
制約	$\mathcal{L}_c = \dfrac{1}{n_r}\sum_{i=1}^{n_r} \mathrm{ReLU}\left(g(t_i)\right)^2$	$g \le 0$ を超えた分だけ罰する

問題3: 同定

問題3

曲率が弧長に比例する曲線の始点側で，n_d 点の座標を測量した (測量誤差 σ)．このとき，パラメータ A と，長さ L の区間全体の経路 $(x(s), y(s))$ を求めなさい．

曲率　$\kappa(s) = \dfrac{s}{A^2}$
始点　$(x, y, \varphi)(0) = (0, 0, 0)$
観測　$s_k = 0, 10, \ldots, 40$ m で (x_k, y_k)
定数　$n_d = 5$，$\sigma = 5$ cm，$L = 80$ m

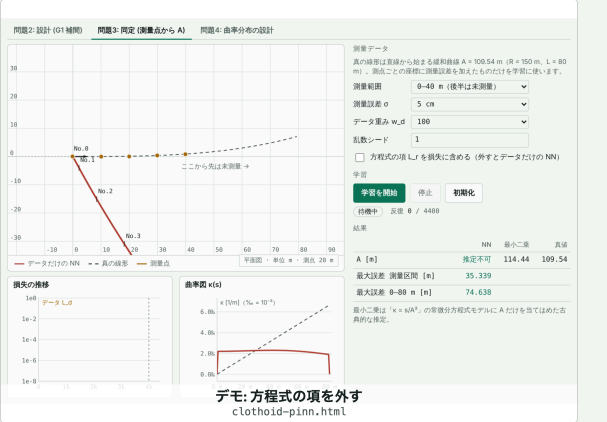


問題3 の定式化

項目	内容
学習器	MLP $f_\theta: t \mapsto (x, y, \varphi)$ (問題1 と同じ)
学習する量	NN のパラメータ θ と物理パラメータ A (未知の係数)
方程式 $\mathcal{L}_r$	曲率 $\kappa = tL/A^2$
境界 $\mathcal{L}_b$	始点: 構造 (終点の条件はない)
データ $\mathcal{L}_d$	測点 s_k (n_d 点) での位置のずれ
損失	$\mathcal{L} = \mathcal{L}_r + w_d\,\mathcal{L}_d,\ \ w_d = 10,\ 100,\ 1000$

- 比較: データのみの NN ($\mathcal{L} = \mathcal{L}_d$)，非線形最小二乗 ($A$ を変えて積分した曲線を n_d 点に当てはめる)

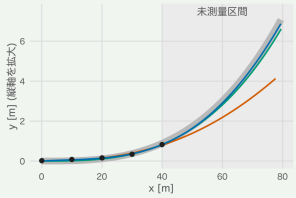
デモ: 問題3 を解く
clothoid-pinn.html



デモ: 方程式の項を外す

clothoid-pinn.html

問題3 の結果



測点 0〜40 m の 5 点 (σ = 5 cm) から 80 m まで再現 (乱数シード 0)

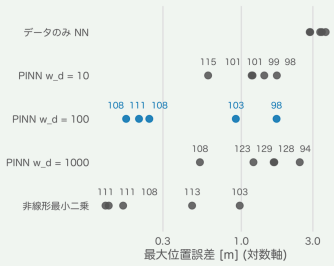
最大位置誤差と A (5 試行)

手法	誤差 [m]	A [m]
真の線形	—	109.5
データのみ NN	2.86〜3.65	—
非線形最小二乗	0.12〜0.98	102.7〜113.4
PINN (w_d = 100)	0.17〜1.72	98.3〜110.9

- データのみ NN は外挿できず，A も推定不可

物理法則の残差が観測のない区間を埋める (正則化としての物理)

問題3 の考察: 損失の重み



最大位置誤差．点は 5 試行，数字は推定した A [m]

- $w_d = 100$ で 0.17〜1.72 m (5 試行中 3 試行は 0.25 m 以下)
- $w_d = 10$ では物理寄りになり 0.6〜1.7 m
- $w_d = 1000$ では 5 試行中 3 試行が $A \approx 122$〜129 m の **局所解** (近くでは最良だが全体では最良でない解．真値 109.5 m)

重みの選び方は PINN の実用上の最大の論点の 1 つ

問題4: 曲率分布の設計

問題4

長さ L の曲線で，両端の位置・方位・曲率と最小半径 $R_{\min}$ を守り，曲率の変化が最も少ない曲率 $\kappa(s)$ と経路を求めなさい．

目的 $\min_{\kappa} \int_0^L |\kappa''(s)|\, ds$

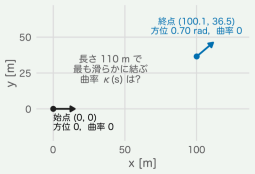
制約 $|\kappa(s)| \le 1/R_{\min}$

始点 $(x, y, \varphi, \kappa)(0) = (0, 0, 0, 0)$

終点 $(x, y, \varphi, \kappa)(L) = (x_f, y_f, \varphi_f, 0)$

定数 $L = 110$ m，$R_{\min} = 100$ m

$(x_f, y_f, \varphi_f) = (100.1\text{ m}, 36.5\text{ m}, 0.70\text{ rad})$



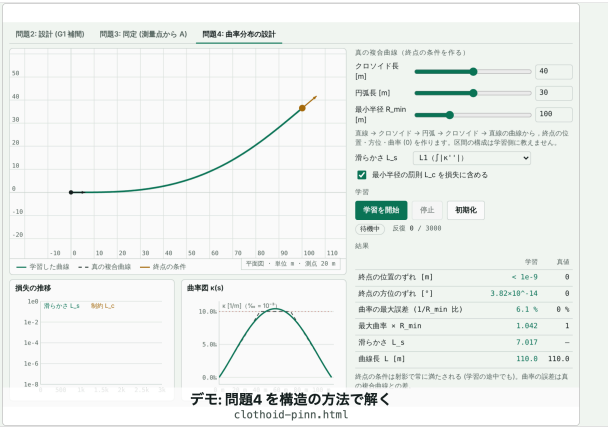
- κ'' の L1 罰則 → 曲率は区分線形 (直線・クロソイド・円弧)

問題4 の定式化

項目	罰則 (問題2 の方法)	構造 (ハード制約)
学習器	MLP $f_\theta: t \mapsto (x, y, \varphi, \kappa)$	MLP $f_\theta: t \mapsto j$ $(j = d\kappa/dt)$．後ろに積分層 $j \to \kappa \to \varphi \to (x, y)$ と射影 $u_\theta = \Phi(f_\theta)$
学習する量	NN のパラメータ θ	NN のパラメータ θ
方程式 $\mathcal{L}_r$	使う	0 (積分層で構造的に満たす)
境界 $\mathcal{L}_b$	始点・終点 (各 4 条件)	射影で厳密に満たす
データ $\mathcal{L}_d$	なし	なし
滑らかさ $\mathcal{L}_s$	使う	使う
制約 $\mathcal{L}_c$	最小半径 $g = R_{\min}\lvert\kappa\rvert - 1$	同左
損失	$\mathcal{L}_r + w_b\mathcal{L}_b + w_s\mathcal{L}_s + w_c\mathcal{L}_c$	$\mathcal{L}_s + w_c\mathcal{L}_c$

- **射影**: j に低次のルジャンドル多項式の補正を足し，終点の 4 条件を Levenberg–Marquardt 法で満たす (微分可能)[1]
- **重み**: $(w_b, w_s, w_c) = (10, 10^{-4}, 10^5)$．右は終点・運動学の重みが要らない

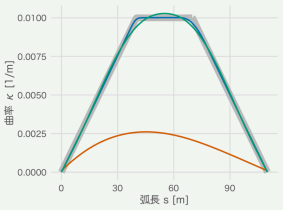
1. ルジャンドル多項式: 区間 [−1, 1] で直交する多項式の族．Levenberg–Marquardt 法: 非線形最小二乗の反復解法 (Gauss–Newton 法と勾配法の中間)



デモ: 問題4 を構造の方法で解く

clothoid-pinn.html

問題4 の結果



求めた曲率分布 (灰色の帯は真値)

終点誤差と曲率の誤差

手法	終点 [m]	κ 誤差
真値	—	—
罰則	0.041	79.2%
ハード +L1	< 1e-9	3.4%
ハード +L2	< 1e-9	5.7%

- 罰則だけでは形が崩れる．L1 は折れ点を保ち，L2 は角を丸める

物理法則を学習器の構造で表現すると，終点・運動学の重みの調整なしに区間構成まで再現できる

問題5: 条件付き PINN

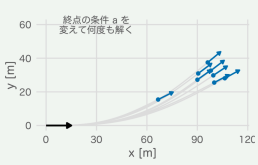
問題5

問題4 の終点の条件 a がさまざまに与えられたとき，どの a に対しても問題4 の解を短い時間 τ で返す写像を求めなさい．

条件 $a = (x_f, y_f, \varphi_f, L)$

写像 $a \longmapsto \kappa(\cdot\,; a)$

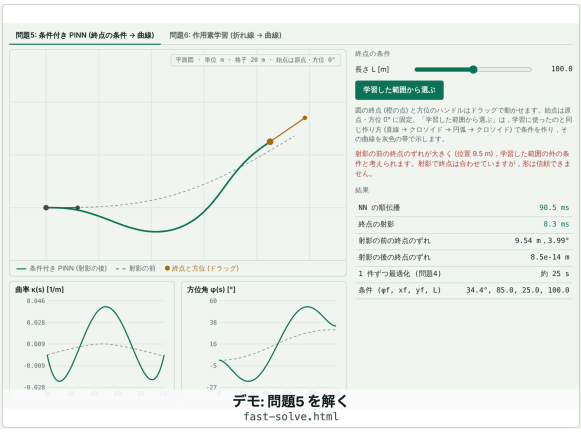
時間 $\tau \approx 1$ ms (1 件あたり)



問題5 の定式化

項目	内容
学習器	MLP $f_\theta\colon (t,a)\mapsto j$ (問題4 と同じ j)．後ろに積分層，射影は推論時だけ
学習する量	NN のパラメータ θ (条件 $a=(\varphi_f,\, x_f/L,\, y_f/L,\, L/100)$ は入力)
方程式 $\mathcal{L}_r$	0 (積分層で構造的に満たす)
境界 $\mathcal{L}_b$	学習中: 罰則，推論時: 射影で厳密に
データ $\mathcal{L}_d$	なし (曲率の正解は使わない)
滑らかさ $\mathcal{L}_s$	使う
損失	$\mathcal{L}=\mathbb{E}_a\left[\mathcal{L}_s+w_b\mathcal{L}_b\right]$ (条件の分布で平均)，$w_b=1000$

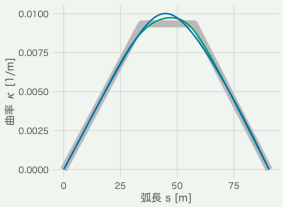
- 問題4 からの変更: 条件 a を入力に加え，条件の族全体で平均した損失を学ぶ (PINO 型)
- 学習中に射影を入れると勾配が不安定 → 学習は罰則，推論時だけ射影



デモ: 問題5 を解く

fast-solve.html

問題5 の結果



未学習の条件での曲率の例

方法	時間/件	$\mathcal{L}_s$ の比
真値	—	—
1 件ずつ最適化	約 25 s	1 (基準)
条件付き PINN	0.7 ms	1.05〜1.5

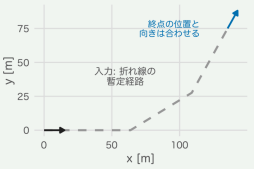
- 時間は推論＋射影．未学習の 200 件すべてで終点が一致．学習 (約 230 s) は一度だけ

「条件 → 解」の写像を前もって学ぶと，1 件ごとの最適化が要らない

問題6: 作用素学習

問題6

折れ線の暫定経路 (方位角の関数 $\varphi_{\mathrm{raw}}(s)$) が与えられたとき，その近くを通り，始点と終点の位置・方位が一致する滑らかな経路の曲率 $\kappa(s)$ を求めなさい．ただし，形も点数も異なる折れ線に対して，一つの写像 $\varphi_{\mathrm{raw}}\mapsto\kappa$ で答えること．



入力　$\varphi_{\mathrm{raw}}(s)$　(折れ線の方位角)

写像　$\varphi_{\mathrm{raw}}\longmapsto\kappa$

終点　$(x,y,\varphi)(L)=(x,y,\varphi)_{\mathrm{raw}}(L)$

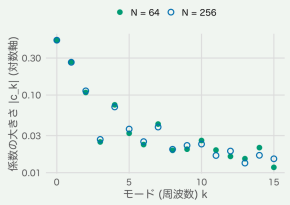
- 注意: 入力の折れ線は **形も点数もまちまち**．決まった個数の数では表せない

問題6 の定式化

項目	内容
学習器	FNO $\mathcal{G}_\theta\colon \varphi_{\mathrm{raw}}(\cdot)\mapsto\hat{\kappa}(\cdot)=L\kappa(\cdot)$ (等間隔の格子 N 点で表す)．後ろに積分層．射影は推論時だけ
学習する量	NN のパラメータ θ (折れ線 φ_{raw} は入力の関数)
方程式 $\mathcal{L}_r$	0 (積分層で構造的に満たす)
境界 $\mathcal{L}_b$	学習中: 罰則，推論時: 射影で厳密に
データ $\mathcal{L}_d$	折れ線 $(x,y)_{\mathrm{raw}}$ からのずれ (正解の経路は使わない)
滑らかさ $\mathcal{L}_s$	使う
損失	$\mathcal{L}=\mathbb{E}_{\varphi_{\mathrm{raw}}}\left[\mathcal{L}_d+w_s\mathcal{L}_s+w_b\mathcal{L}_b\right]$，$(w_s,w_b)=(5\times10^{-6},\,10)$

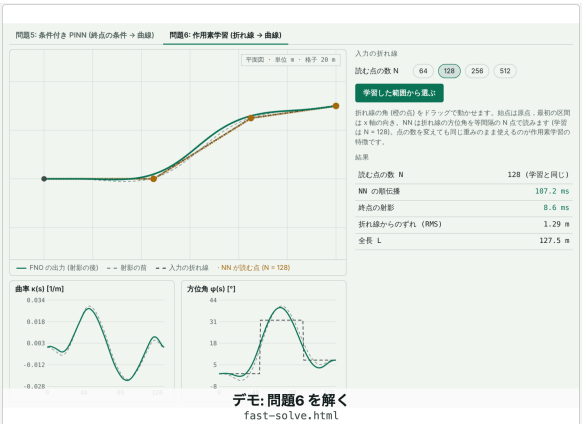
- 問題5 からの変更: 条件が **関数** になる → NO (FNO) で関数から関数への写像を学ぶ．物理の損失を使うので PINO
- FNO: フーリエ空間で低次モードだけを重み付けする層 → 解像度に依存しない

なぜ点数が違ってよいか



同じ折れ線を等間隔の N 点で読んだときのフーリエ係数

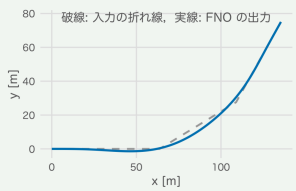
- FNO の重みは格子点ではなく **フーリエモード** (周波数) ごとに付く
- 同じ関数なら，点数 N が違っても低いモードの係数はほぼ同じ (左図)
- だから同じ重みがどの N にも使える．学習は $N=128$，推論は 256・512
- 格子は等間隔 (FFT のため)．折れ線の節点の数とは関係なく，方位角を等間隔の N 点で読む



デモ: 問題6 を解く

fast-solve.html

問題6 の結果



N = 128 で学習し N = 256 で推論した例

未学習の折れ線 200 本 (中央値)

N	ずれ [m]	端処理あり	ms/件
128	1.46	2.10	0.27
256	1.45	2.08	0.30
512	1.45	2.08	1.56

- 学習は N = 128 だけ
- 端の処理 (トレンド除去＋ゼロ埋め) を入れると，かえって悪化

関数から関数への写像を学ぶので，解像度を変えても同じ精度 (**解像度不変**)

この例題から学ぶべきこと

データ: 問題3
- 測量 5 点でも，観測のない区間は方程式が補う
- データだけの NN は 3 m のずれ
- 求める量が数個 (問題3 は A の 1 つ) なら古典の当てはめと同程度

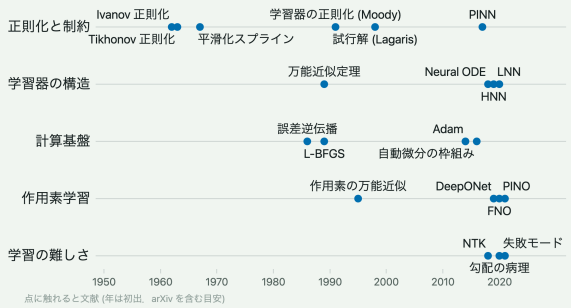
安全: 問題4
- 罰則より構造: 物理と条件を学習器の形に
- 終点のずれ 4 cm → 10^{-3} m 未満
- 重みの調整が不要

速さ: 問題5・6
- 条件 → 解の写像を前もって学習
- 1 件 25 秒 → 0.7 ms
- 入力が関数でも，点数によらず同じ重み

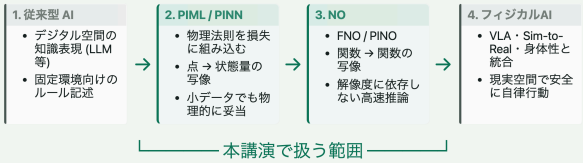
PIML が生きるのは **求めるものが関数のとき** と **条件を変えて何度も解くとき**．確実に計算できる部分は数値解析に任せる

フィジカルAI
物理の組み込み方: 3 つのバイアス
バイアスを実現する技術
例題: 制約付き曲線の設計
▸ 研究の流れとまとめ

技術の年表



研究の流れ



まとめ

1. PIML は古典的な学習 (データ＋正則化＋仮説空間) の **素直な拡張**．物理を組み込む場所は 3 つ
2. 微分・積分を含む条件を学習できるのは **自動微分と数値積分** のおかげ
3. 罰則 (学習バイアス) は手軽，構造 (帰納バイアス) は厳密．問題に応じて **ソフトからハードへ** (問題4: 終点の誤差 4 cm → 10^{-3} m 未満)
4. 関数から作用素へ:「1 回解く」から **「何度でもすぐ解ける」** へ (問題5: 25 秒 → 0.7 ms)
5. 古典数値解析と **分担する**．確実に計算できる部分は数値解析，関数の未知と繰り返しは学習に

参考文献

資料の作成には生成 AI を補助に用いた (2026 年 8〜10 月)。Claude (Anthropic): 構成の検討，文章，数値実験と作図のコード，デモ，Gemini (Google): 文献の探索と要約，質疑応答による内容の検討，内容の責任は著者にある。

Chen, T., & Chen, H. (1995). Universal approximation to nonlinear operators by neural networks with arbitrary activation functions and its application to dynamical systems. *IEEE Transactions on Neural Networks*, 6(4), 911–917. https://doi.org/10.1109/72.392253

Cybenko, G. (1989). Approximation by superpositions of a sigmoidal function. *Mathematics of Control, Signals and Systems*, 2(4), 303–314. https://doi.org/10.1007/BF02551274

Greydanus, S., Dzamba, M., & Yosinski, J. (2019). Hamiltonian Neural Networks. *Advances in Neural Information Processing Systems*, 32. https://proceedings.neurips.cc/paper_files/paper/2019/hash/26cd8ecadac0d4elb6cc8ad8725cbd1b-Abstract.html

Hornik, K., Stinchcombe, M., & White, H. (1989). Multilayer feedforward networks are universal approximators. *Neural Networks*, 2(5), 359–366. https://doi.org/10.1016/0893-6080(89)90020-8

Karniadakis, G. E., Kevrekidis, I. G., et al. (2021). Physics-informed machine learning. *Nature Reviews Physics*, 3(6), 422–440. https://doi.org/10.1038/s42254-021-00314-5

Lagaris, I. E., Likas, A., & Fotiadis, D. I. (1998). Artificial neural networks for solving ordinary and partial differential equations. *IEEE Transactions on Neural Networks*, 9(5), 987–1000. https://doi.org/10.1109/72.712178

Mirjany, A., & Kovač, M. (2020). Skills for physical artificial intelligence. *Nature Machine Intelligence*, 2(11), 658–660. https://doi.org/10.1038/s42256-020-00258-y

Moody, J. (1991). The Effective Number of Parameters: An Analysis of Generalization and Regularization in Nonlinear Learning Systems. *Advances in Neural Information Processing Systems*, 4. https://proceedings.neurips.cc/paper/1991/hash/e65e4a240bcb83f3f36d5fe5fd61874281454-Abstract.html

Nikjar, O., Mahoney, M. W., & Krishnapriyan, A. S. (2023, May). Learning differentiable solvers for systems with hard constraints. *The Eleventh International Conference on Learning Representations*. https://openreview.net/forum?id=vdv6CmOksr0

Raissi, M., Perdikaris, P., & Karniadakis, G. E. (2019). Physics-informed neural networks: A deep learning framework for solving forward and inverse problems involving nonlinear partial differential equations. *Journal of Computational Physics*, 378, 686–707. https://doi.org/10.1016/j.jcp.2018.10.045

Sukumar, N., & Srivastava, A. (2022). Exact imposition of boundary conditions with distance functions in physics-informed deep neural networks. *Computer Methods in Applied Mechanics and Engineering*, 389, 114333. https://doi.org/10.1016/j.cma.2021.114333

Thakur, A., Kalpa, K., et al. (2025). Physical Artificial Intelligence for Powering the Next Revolution in Robotics. *Journal of Computing and Information Science in Engineering*, 25(12), 120809. https://doi.org/10.1115/1.4070122

Tikhonov, A. N., & Arsenin, V. Y. (1977). *Solutions of Ill-posed Problems* (F. John, Trans.). Winston. https://books.google.com/?id=ECnvAAAAMAAJ (Original work published as *Методы решения некорректных задач*, 1974)

Wahba, G. (1990). *Spline models for observational data*. Society for Industrial and Applied Mathematics (SIAM). https://doi.org/10.1137/1.9781611970128

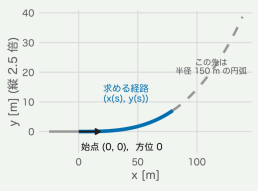
付録: 例題の詳細と課題

問題1: 順問題

問題1

曲率と始点の位置・向きが与えられたとき，経路に沿った弧長 s の関数として経路 $(x(s), y(s), \varphi(s))$ を求めなさい．

曲率　$\kappa(s) = \dfrac{s}{RL} \quad (0 \le s \le L)$
始点　$(x, y, \varphi)(0) = (0, 0, 0)$
定数　$R = 150\ \mathrm{m},\ L = 80\ \mathrm{m}$

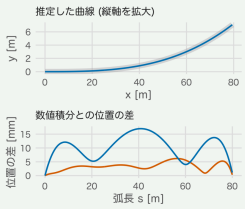


問題1 の定式化

項目	内容
学習器	多層 NN (MLP) $f_\theta: t \mapsto (x, y, \varphi)$．解は $(x, y, \varphi)(t) = t\, f_\theta(t)$
学習する量	NN のパラメータ θ
方程式 $\mathcal{L}_r$	曲率 $\kappa = t/R$
境界 $\mathcal{L}_b$	始点: 構造 (ハード)．比較として罰則 (ソフト) も
データ $\mathcal{L}_d$	なし
損失	$\mathcal{L} = \mathcal{L}_r$ (ソフトは $\mathcal{L}_r + w_b\mathcal{L}_b$)，$w_b = 10$

- $t = s/L \in [0, 1]$: 正規化した弧長 (NN の入力)
- 各項の具体形は「例題の損失関数」の頁

問題1 の結果



	方法	最大誤差	時間
	PINN (ハード)	6.4〜17.0 mm	約 4 s
	PINN (ソフト)	3.3〜9.1 mm	約 4 s
	数値積分	0.02 mm	0.3 ms

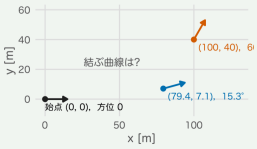
- PINN は乱数シード 5 通りの範囲
- 精度は十分だが，時間は数値積分の 1 万倍

方程式も入力も分かっている順問題なら数値積分で十分．PINN が生きるのは未知のものが増えたとき

問題2: 設計

問題2

曲率が弧長に比例して変わる曲線について，始点と終点の位置・方位が与えられたとき，両者を結ぶ曲線の始点と終点の曲率 κ_0, κ_1 と長さ L を求めなさい．



端点の位置と向きを指定して曲線で結ぶ (G1 エルミート補間)[1]

曲率　$\kappa(s) = \kappa_0 + (\kappa_1 - \kappa_0)\,\dfrac{s}{L}$

始点　$(x, y, \varphi)(0) = (0, 0, 0)$

終点　$(x, y, \varphi)(L) = (x_1, y_1, \varphi_1)$

定数　$(x_1, y_1, \varphi_1) = (79.4\ \text{m},\ 7.1\ \text{m},\ 15.3^\circ)$ または $(100\ \text{m},\ 40\ \text{m},\ 60^\circ)$

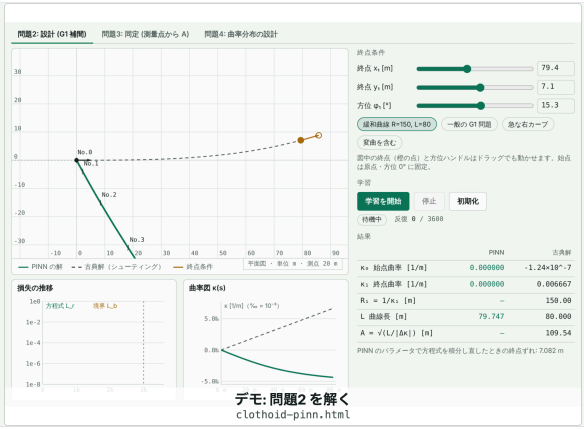
1. G1 (幾何学的連続の 1 次): 曲線どうしが位置と接線の向きをそろえてつながること (G0 は位置だけ，G2 は曲率まで)

問題2 の定式化

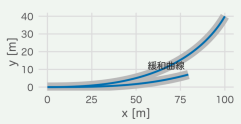
項目	内容
学習器	MLP $f_\theta: t \mapsto (x, y, \varphi)$ (問題1 と同じ)
学習する量	NN のパラメータ θ と物理パラメータ $\lambda = (\kappa_0, \kappa_1, L)$ (設計変数)
方程式 $\mathcal{L}_r$	曲率 $\kappa = \kappa_0 + (\kappa_1 - \kappa_0)t$
境界 $\mathcal{L}_b$	始点: 構造，終点: 目標 (x_1, y_1, φ_1) とのずれ
データ $\mathcal{L}_d$	なし
損失	$\mathcal{L} = \mathcal{L}_r + w_b\mathcal{L}_b$，$w_b = 10$

- 問題1 からの変更: λ も学習する．終点の条件を罰則 (ソフト) で加える
- 参照解: ODE の数値積分＋Newton 法 (シューティング)．Adam 3,000 回 → L-BFGS 300 回[1]

1. Adam: 勾配の大きさを自動で調整する一次の最適化法．L-BFGS: 曲率 (2 階微分) の近似を使う準ニュートン法．シューティング法: 始点側の未知量を変えて積分し，終点の条件が合うよう Newton 法で調整する方法



問題2 の結果



求めた物理パラメータ (乱数シード 0)

	手法	κ_0 [1/m]	κ_1 [1/m]	L [m]
	緩和 古典	0	0.0066667	80.000
	緩和 PINN	3e-07	0.0066662	80.000
	G1 古典	0.0018108	0.0167232	113.003
	G1 PINN	0.0018113	0.0167226	113.003

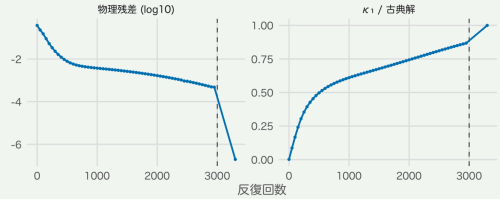
- 終端条件 (x_1, y_1, ϕ_1): 緩和曲線 (79.4, 7.1, 15.3°)，G1 (100, 40, 60°)[1]

終端条件だけから求めた曲線 (灰色の帯: 古典解，青線: PINN)

終端条件だけから緩和曲線 ($\kappa_0 \approx 0$，$R = 150$ m，$L = 80$ m) を古典解と 4 桁以上の一致で復元

1. 緩和曲線: 直線と円弧の間に入れて曲率を徐々に変える曲線 (道路ではクロソイド)．ここでは問題1 の曲線の終点を与えた場合．一般の G1: 任意の終点の位置・向きに合わせる問題

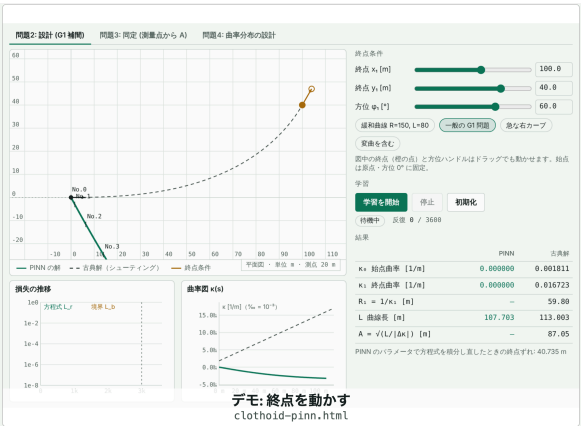
問題2 の考察: 学習の経過

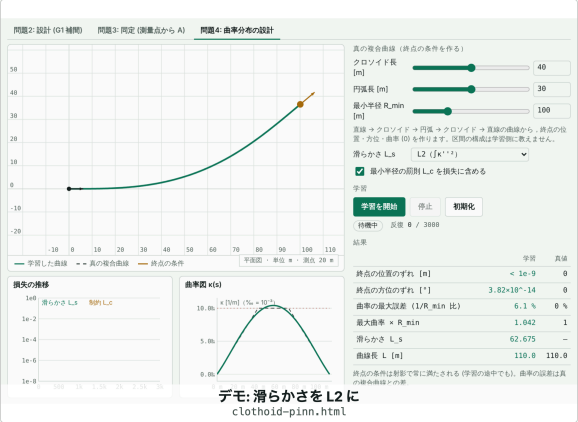


学習の経過 (一般の G1 問題)．破線で Adam から L-BFGS に切替

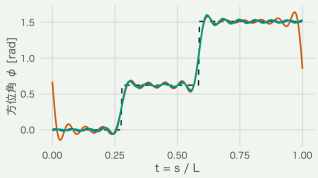
- Adam 2,700 回の時点でも κ_1 は古典解の 84% に留まり，L-BFGS で一気に収束した

損失の谷が細長い (悪条件) ので，曲率の情報を使う二次の方法が効く





問題6 の補足: ギブス現象



折れ線の方位角を 16 モードで打ち切ったフーリエ級数で表す

両端 5% の最大誤差

方法	誤差 [rad]
元の折れ線	—
そのまま	0.673
トレンド除去	0.015
反射拡張	0.020

- FFT は周期とみなすので，両端の段差の約 45% が振動に (**ギブス現象**[1])
- トレンド除去 (両端を結ぶ直線を引く)・反射拡張 (折り返して偶関数に) で 1/30 以下

例題の出力 (曲率) は両端に段差がなく，端の処理は不要

1. 段差 (不連続) のある関数を打ち切ったフーリエ級数で表すと，段差の近くに項数を増やしても消えない振動 (行き過ぎ) が出る現象

速度と幾何

- 速度 v で曲がると

$$a_y = v^2\kappa$$
$$j_y = v^3\frac{d\kappa}{ds} + 2v\,a_t\,\kappa$$

- 問題4 の幾何 (R = 100 m) を 20 m/s で走ると $a_y = 4.0$ m/s², $j_y = 2.0$ m/s³
- 加速度の上限: 横 3.0，摩擦円 3.5 m/s²
- 横躍度の上限: 0.8 m/s³
- 摩擦円: タイヤが出せる加速度 (前後と横の合成) の上限，横躍度 (横ジャーク): 横加速度の変化率 (乗り心地に効く)

分離型 (7a): 減速して守る

幾何は固定し，速度 $v(s)$ を凸最適化で決める

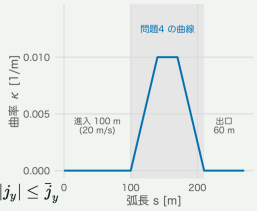
統合型 (7b): 速度を保って守る

動特性の罰則を損失に入れ，幾何そのものを変える

問題7a: 速度の最適化

問題7a

進入直線・問題4 の曲線・出口直線をつないだ経路の曲率と，進入速度 v_0，加速度と躍度の上限が与えられたとき，これらを守り走行時間 T が最短となる速度 $v(s)$ を求めなさい．



目的 $\min_v T = \int \frac{ds}{v(s)}$

制約 $|a_y| \le \bar{a}_y,\ |a_t| \le \bar{a}_t,\ \sqrt{a_t^2 + a_y^2} \le \bar{a},\ |j_y| \le \bar{j}_y$

始点 $v(0) = v_0$

定数 $L_{\text{in}} = 100$ m, $L = 110$ m, $L_{\text{out}} = 60$ m, $v_0 = 20$ m/s

$(\bar{a}_y, \bar{a}_t, \bar{a}) = (3.0, 2.0, 3.5)$ m/s², $\bar{j}_y = 0.8$ m/s³

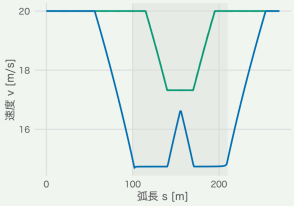
問題7a の定式化

項目	内容
学習器	なし (NN は使わない)
求める量	速度の 2 乗 $b = v^2$ の格子点の値 $b_1, \dots, b_n$ ($\Delta s = 1$ m, $n = 271$)
方程式	$a_y = b\,\kappa$, $a_t = b'/2$ (b について線形)
境界	$b_1 = v_0^2$
データ	なし
制約 (凸)	$\lvert a_y\rvert \le \bar{a}_y$, $\lvert a_t\rvert \le \bar{a}_t$, $\sqrt{a_t^2 + a_y^2} \le \bar{a}$ (二次錐)
制約 (非凸)	$\lvert j_y\rvert \le \bar{j}_y$ (線形化して解き直す: 逐次凸化)
目的	$T = \int b^{-1/2}\,ds$ を最小 (凸)

- $b = v^2$ と置くと横加速度・摩擦円は凸 → 二次錐計画 (**SOCP**)．横躍度は線形化して解き直す (**逐次凸化**)[1]
- NN は使わない: 幾何が決まれば，速度は古典的な最適化で大域最適に解ける

1. SOCP (二次錐計画): 線形の目的と二次錐の制約をもつ凸最適化で，大域最適解が効率よく求まる．逐次凸化: 非凸の制約を現在の解のまわりで線形化した凸問題を繰り返し解く方法

問題7a の結果



問題4 の幾何 (網掛け) を走る最短時間の速度

走行時間と計算時間

制約	T [s]	j_y	ms
a_y のみ	14.0	2.00	35
＋摩擦円	14.0	2.00	23
＋j_y	16.0	0.80	281

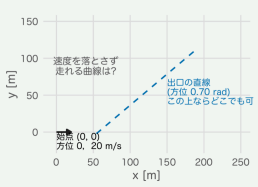
- 曲率が変わるクロソイド区間でさらに減速が要る

幾何を固定すると，減速して制約を守るしかない (走行時間 14.0 → 16.0 s)

問題7b: 幾何と速度の統合

問題7b

始点の位置・方位・曲率と出口の直線 (問題 4 の終点を通り方位 φ_f) が与えられたとき，一定の速度 v のまま横加速度と横躍度の上限を守って走れ，出口の直線に方位を合わせて曲率 0 で入る曲線の曲率 $\kappa(s)$ と長さ L を求めなさい．



制約 $\left|v^2\kappa\right| \le \bar{a}_y,\ \left|v^3\frac{d\kappa}{ds}\right| \le \bar{j}_y$

始点 $(x, y, \varphi, \kappa)(0) = (0, 0, 0, 0)$

終点 $\varphi(L) = \varphi_f,\ \kappa(L) = 0,\ (x, y)(L) \in$ 出口の直線

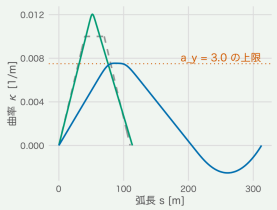
定数 $v = 20$ m/s, $\varphi_f = 0.70$ rad ($\bar{a}_y, \bar{j}_y$ は 7a と同じ)

問題7b の定式化

項目	内容
学習器	MLP $f_\theta: t \mapsto j$．後ろに積分層と射影 (問題4 と同じ)
学習する量	NN のパラメータ θ と物理パラメータ L (長さ)
方程式 $\mathcal{L}_r$	0 (積分層で構造的に満たす)
境界 $\mathcal{L}_b$	射影: $\kappa(1) = 0,\ \varphi(1) = \varphi_f$，出口の直線までの距離 0
データ $\mathcal{L}_d$	なし
滑らかさ $\mathcal{L}_s$	使う
制約 $\mathcal{L}_c$	動特性 $g = \left(v^2\lvert\kappa\rvert/\bar{a}_y - 1,\ v^3\lvert d\kappa/ds\rvert/\bar{j}_y - 1\right)$
損失	$\mathcal{L} = \mathcal{L}_s + w_c\mathcal{L}_c + w_\ell L,\ (w_c, w_\ell) = (10^4, 0.02)$

- 問題4 からの変更: 動特性の上限を超えた分だけ罰則 ($\mathcal{L}_c$)．$w_\ell L$ は曲線を短くする圧力
- 7a と逆: 速度を保ち，幾何の方を変える

問題7b の結果



同じ角 (40°) を 20 m/s のまま曲がる軌道の曲率

20 m/s で走ったときの値

	損失	L	a_y	j_y
----	問題4 の幾何	110	4.00	2.00
——	幾何のみ	113	4.81	1.92
——	＋動特性	313	3.02	0.80

- a_y, j_y は上限付近まで下がり，曲線長は 3 倍に

動特性を損失に入れると，速度を落とさずに走れる幾何が得られる

課題と研究動向

課題	例題で見えたこと	主な対策
損失の重みと最適化	重みで結果が大きく変わる．収束が遅い	重みの自動調整，Adam → L-BFGS，無次元化
制約の厳密さ	罰則では終点や上限をわずかに破る	積分層・射影 (構造で満たす)，凸最適化との分担
計算コスト	1 件ごとの学習は古典解法より遅い	条件付き PINN・作用素学習 (学習は一度，推論は速い)
表現の限界	開曲線ではフーリエ層の端が振動しうる	端の処理，何を出力にするかの選択

PIML と古典数値解析の使い分け

古典数値解析に任せる

- 方程式も入力も分かった問題を 1 つ速く正確に解く
- 未知数が数個の設計・同定
- 制約を厳密に守る (凸最適化なら大域最適)

学習に任せる

- 未知が **関数** で，解の形を仮定できない
- 条件を変えて **何度も** 解く
- 観測と方程式を同じ損失で組み合わせる

分担の原則

確実に計算できる部分 (積分・射影・凸最適化) は数値解析に任せ，学習には **関数の未知** と **繰り返し** を任せる